Intelligent Diabetes Risk Predictor

Detailed Project Understanding Guide

CS619 Final Project

Prepared: June 24, 2026

This document explains the complete Intelligent Diabetes Risk Predictor system — what it does, how it is built, how each role uses it, and how the machine learning pipeline works from data input to risk prediction.

Important: This is a **decision-support** tool for education and risk awareness. It is **not** a medical diagnosis. Always consult a qualified healthcare professional for clinical decisions.

1. Introduction

1.1 What Is This Project?

The Intelligent Diabetes Risk Predictor is a full-stack web application that uses machine learning to estimate a person's risk of developing diabetes based on clinical and lifestyle measurements. It was developed as the CS619 Final Project under supervision of Komal Khawer (komal.khawer@vu.edu.pk).

Unlike a simple Python script that only prints a prediction, this is a complete system with user accounts, role-based access, a database, security controls, dashboards, report export, education content, clinical workflows, and cloud deployment.

1.2 Why This Project Matters

- Diabetes affects millions worldwide; early detection enables prevention.
- Multiple health factors (glucose, BMI, blood pressure, age, etc.) interact in complex ways — ML can analyze them together.
- A web interface makes risk assessment accessible to patients at home.
- Doctors can use it as clinical decision support for assigned patients.
- Admins can manage models, data, and system quality over time.

1.3 What the System Is NOT

- It does not replace a doctor or laboratory diagnosis.
- It does not use real-time hospital equipment or EHR integration.
- It is trained on a historical research dataset (Pima Indians), not your local population.
- Predictions are probability estimates, not certainties.

2. System Overview

2.1 Three User Roles

Patient — End users who enter their health data, receive risk predictions, track progress over time, read education content, submit feedback, and export reports.

Healthcare Provider (Doctor) — Clinicians who view assigned patients only, review predictions and trends, add clinical notes, use decision-support tools, and respond to cases forwarded by the administrator.

Administrator — System managers who control users, datasets, model training, education content, feedback review, audit logs, and the clinical escalation workflow.

2.2 Core Capabilities (19 CS619 Requirements)

- User management with role-based access control (Patient, Provider, Admin)
- Health data input and admin dataset import
- Exploratory data analysis (EDA) with charts
- Data preprocessing, scaling, and 70/30 train-test split
- Four ML models: Neural Network, SVM, Decision Tree, Logistic Regression
- Evaluation metrics: Accuracy, Precision, Recall, F1, Confusion Matrix
- Model comparison, auto-selection, and admin override
- Model persistence with joblib
- ML-based risk factor analysis and clinical threshold alerts
- Personalized recommendations by risk level
- Longitudinal tracking with trend alerts
- Role-specific dashboards with Chart.js
- PDF and CSV report export
- Education resources with admin content management
- Clinical decision support and provider notes
- Feedback collection and model retraining from verified outcomes
- Admin panel: users, assignments, training history, audit logs
- Performance optimization: model caching, fast inference
- Security: password hashing, CSRF, rate limiting, audit trail, consent

3. System Architecture

3.1 Three-Tier Design

The application follows a classic three-tier monolithic architecture:

- **Presentation Tier:** Jinja2 HTML templates, Bootstrap CSS, Chart.js charts, static plot images.
- **Application Tier:** Flask web framework with blueprints (auth, main, admin, provider), WTForms validation, Flask-Login sessions.
- **Data & ML Tier:** SQLAlchemy ORM + SQLite/PostgreSQL database, CSV dataset in data/, trained models in saved_models/.

3.2 Flask Blueprints

- **auth_bp** — Login (patient/doctor/admin portals), registration, profile, logout.

• **main_bp** — Patient dashboard, health data entry, predictions, progress, education, feedback, reports.

• **admin_bp** (prefix /admin) — Dataset upload, EDA, model training, user management, assignments, CMS, audit.

• **provider_bp** (prefix /provider) — Doctor dashboard, patient detail, clinical support, forwarded reports.

3.3 Application Factory (create_app)

The app is created via create_app() in app/__init__.py. On startup it:

• Loads configuration from config.py (database URI, folders, security settings).

• Creates required folders: data/, saved_models/, app/static/plots/, instance/.

• Initializes SQLAlchemy and Flask-Login.

• Registers all blueprints and security headers.

• Runs database schema migrations for new columns.

• Seeds default users, education resources, and owner access codes.

• Regenerates missing EDA plots if the dataset exists.

• Clears the ML model cache so fresh models are loaded.

3.4 Project Folder Structure

• app/routes/ — auth.py, main.py, admin.py, provider.py

• app/ml/ — pipeline.py, predictor.py, explainability.py, recommendations.py, retrain.py, reports.py

• app/templates/ — HTML pages for all roles

• app/static/ — CSS and generated plot images

• app/models.py — Database table definitions

• app/forms.py — WTForms with validation rules

• app/security.py — Rate limiting and HTTP security headers

• data/ — diabetes.csv dataset

• saved_models/ — Trained .joblib model files

• instance/ — SQLite database (local dev)

• docs/ — SRS, User Manual, Test Plan, PDFs

• tests/ — pytest unit tests

• config.py — App configuration

• run.py — Development server entry point

• train_initial.py — First-time model training script

• setup_data.py — Dataset download script

• Dockerfile — Production container

• render.yaml — Render cloud deployment blueprint

4. Technology Stack

- **Python 3.12** — Primary programming language.
- **Flask 3.x** — Lightweight web framework for routing, templates, and sessions.
- **Flask-SQLAlchemy** — Object-relational mapping for database tables.
- **Flask-Login** — Session-based authentication and current_user management.
- **Flask-WTF / WTForms** — Form handling with CSRF protection and validation.
- **scikit-learn** — ML algorithms, preprocessing, metrics, and pipelines.
- **pandas / numpy** — Data manipulation for training and inference.
- **matplotlib / seaborn** — EDA plots, confusion matrices, model comparison charts.
- **joblib** — Serialize and load trained ML pipelines.
- **ReportLab** — Generate PDF reports for patients and admins.
- **Chart.js** — Interactive charts on dashboards and progress page.
- **Bootstrap** — Responsive UI layout and styling.
- **Gunicorn** — Production WSGI server inside Docker.
- **SQLite / PostgreSQL** — Local dev vs. production persistent database.
- **Render + Docker** — Cloud hosting with health checks.

5. Database Design

5.1 Main Tables

users — Stores accounts: username, email, hashed password, full name, role (patient/provider/admin), active status.

health_records — Patient vitals per entry: sex, pregnancies, glucose, systolic/diastolic BP, skin thickness, insulin, BMI, diabetes pedigree, age, timestamp.

predictions — ML output linked to a health record: model name, probability, risk level (Low/Moderate/High), explanation JSON, recommendations JSON.

model_metrics — Training results per model: accuracy, precision, recall, F1, confusion matrix, is_best flag.

training_jobs — Metadata for each training run: type, rows used, feedback rows, best model, status.

feedbacks — User ratings and actual clinical outcomes for retraining.

provider_patients — Many-to-many link between doctors and their assigned patients.

clinical_notes — Doctor notes attached to a patient prediction.

education_resources — CMS articles: title, category, content, optional external URL.

audit_logs — Security trail: user, action, resource, IP address, timestamp.

admin_report_submissions — Patient reports sent to admin for review.

doctor_report_forwards — Admin forwards patient reports to assigned doctors.

doctor_report_remarks — Doctor remarks sent back to patients.

5.2 Key Relationships

- One User has many HealthRecords and Predictions.
- Each HealthRecord generates one Prediction.
- Each Prediction can have Feedback entries.
- ProviderPatient links doctors to patients they are allowed to view.
- TrainingJob records produce multiple ModelMetrics rows.

5.3 Database Configuration

Locally, SQLite stores data at instance/diabetes.db. In production, set the DATABASE_URL environment variable to a PostgreSQL connection string (e.g. from Neon). Without PostgreSQL on Render, data may reset when the app redeploys.

6. Machine Learning Pipeline

6.1 Dataset

The system uses the Pima Indians Diabetes Dataset (UCI / Kaggle diabeticprediction). It contains 8 input features and a binary outcome (0 = no diabetes, 1 = diabetes):

- Pregnancies — Number of times pregnant
- Glucose — Plasma glucose concentration (mg/dL)
- BloodPressure — Diastolic blood pressure (mm Hg)
- SkinThickness — Triceps skin fold thickness (mm)
- Insulin — 2-hour serum insulin (mu U/ml)
- BMI — Body mass index (kg/m²)
- DiabetesPedigreeFunction — Family history score
- Age — Age in years

6.2 Data Preprocessing

- Zeros in Glucose, BloodPressure, SkinThickness, Insulin, and BMI are treated as missing values (common in this dataset).
- Missing values are imputed using the median (SimpleImputer).
- All features are standardized with StandardScaler (zero mean, unit variance).
- Data is split 70% training / 30% testing with stratification to preserve class balance.
- Random state is fixed at 42 for reproducible results.

6.3 Four ML Models

- **Neural Network (MLPClassifier):** Hidden layers (32, 16), ReLU activation, early stopping.
- **SVM (SVC):** RBF kernel, probability enabled, balanced class weights.
- **Decision Tree:** Max depth 5, balanced class weights.
- **Logistic Regression:** Max 2000 iterations, balanced class weights.

Each model is wrapped in a scikit-learn Pipeline: Imputer → Scaler → Classifier. This ensures the same preprocessing is applied during both training and live prediction.

6.4 Model Evaluation & Selection

After training, each model is evaluated on the 30% test set:

- **Accuracy** — Overall correct predictions.
- **Precision** — Of predicted diabetics, how many are actually diabetic.
- **Recall** — Of actual diabetics, how many the model detected.
- **F1-Score** — Harmonic mean of precision and recall (used for comparison).
- **ROC-AUC** — Area under the receiver operating characteristic curve.
- **Confusion Matrix** — Visual heatmap saved as PNG per model.

The model with the highest ROC-AUC is marked as best. A separate production pipeline (default: Logistic Regression) is trained on all data for patient-facing predictions. Admin can override the production model via Model Settings.

6.5 Live Prediction Flow

- Patient submits the health data form.
- Form values are saved as a HealthRecord in the database.
- record_to_frame() converts the record to a pandas DataFrame matching training features.
- Systolic and diastolic BP are combined into mean arterial pressure: (2×systolic + diastolic) / 3.
- Family history yes/no is mapped to pedigree values: 0.28 (no) or 0.65 (yes).
- The saved production pipeline loads from saved_models/ (cached in memory).
- predict_proba() returns the diabetes probability.
- Risk level is assigned: Low (<35%), Moderate (35–65%), High (>65%).
- explain_prediction() generates feature importance + clinical threshold alerts.
- generate_recommendation_plan() creates personalized lifestyle and medical advice.
- Results are saved in the predictions table and displayed to the patient.

6.6 Explainability

The system uses a hybrid explanation approach:

- **ML Feature Importance:** Top contributing features from model coefficients (logistic regression) or feature importances (decision tree).
- **Clinical Thresholds:** Checks if values exceed medical guidelines — e.g. glucose > 140, BMI > 30, age > 45, systolic > 140.

Both are combined into a JSON explanation shown on the prediction result page.

6.7 Recommendations

Recommendations are structured by risk level:

- **Low risk:** Maintain healthy habits, annual checkup, balanced diet, 150 min/week exercise.

- **Moderate risk:** Doctor appointment within 2–4 weeks, fasting glucose and HbA1c tests, diet changes, daily walking.
- **High risk:** Urgent medical consultation, strict diet control, blood sugar monitoring, weight management.

Additional targeted advice is added based on which specific features are elevated (high glucose, high BMI, etc.).

6.8 Model Retraining

Patients and providers can submit feedback with the actual clinical outcome (diabetic or not). Admin reviews feedback and triggers retraining. Verified feedback rows are merged with the original dataset, models are retrained, and new metrics are stored in training_jobs and model_metrics.

7. Features by Role

7.1 Patient Features

- Register with consent checkbox / Login at /login/patient
- Dashboard with latest prediction, trend chart, and risk alert
- Health Data form — enter vitals with inline help (averages, units, clinical ranges)
- Instant ML prediction with risk level, explanation, and recommendations
- My Health Records — view and edit past entries (re-triggers prediction)
- Progress page — Chart.js line chart of risk probability over time
- Education — browse prevention and lifestyle articles
- Feedback — rate prediction accuracy, submit actual clinical outcome
- Send Report to Admin — escalate high-risk cases with a message
- Notifications — read doctor remarks on your reports
- Export PDF and CSV health reports

7.2 Doctor (Provider) Features

- Login at /login/doctor with owner access code
- Clinical dashboard — assigned patients, high-risk cases highlighted
- Patient detail — health records, predictions, trend charts
- Clinical Support — ML risk analysis view + add clinical notes
- Forwarded Reports — cases sent by admin with admin notes
- Add remarks to patients (patient receives notification)
- Export patient reports as PDF or CSV
- Submit feedback on prediction accuracy

7.3 Admin Features

- Login at /login/admin with owner access code
- Admin dashboard — system statistics and quick links
- Upload Dataset — import CSV with required columns
- EDA — summary statistics, correlation heatmap, histograms, outcome distribution
- Train Models — train all 4 algorithms, view comparison chart
- Model Settings — set which model is used for live patient predictions
- Retrain from Feedback — incorporate verified patient outcomes
- User Management — create, edit roles, activate/deactivate, reset passwords
- Assignments — link doctors to patients
- Education CMS — add, edit, delete prevention articles
- Feedback Review — read patient feedback, mark as read
- Received Reports — triage patient reports, forward to doctors
- Training History — view past training jobs and metrics

• Audit Logs — monitor security-sensitive actions

8. Security Design

• **Password Hashing:** Werkzeug generate_password_hash — passwords never stored in plain text.

• **CSRF Protection:** Flask-WTF tokens on every form submission.

• **Role-Based Access Control:** @role_required decorator blocks unauthorized route access.

• **Owner Access Codes:** Admin and doctor logins require an extra private code beyond password.

• **Login Rate Limiting:** Max 5 failed attempts per IP+username in 5 minutes.

• **Session Security:** HttpOnly cookies, SameSite=Lax, Secure flag on production.

• **HTTP Security Headers:** X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy.

• **Audit Logging:** Login, predictions, training, user changes recorded with IP.

• **Consent:** Required checkbox on patient registration.

• **Data Isolation:** Doctors only see assigned patients; patients only see own data.

9. Key Workflows

9.1 Patient Prediction Workflow

• Patient logs in → opens Health Data page

• Enters vitals (glucose, BP, BMI, age, etc.) with form validation

• System saves HealthRecord → runs ML pipeline → saves Prediction

• Patient sees risk level, probability, top risk factors, and recommendations

• Patient can view progress over time, export report, or send to admin

9.2 Admin Model Training Workflow

• Admin uploads or uses existing diabetes.csv dataset

• Views EDA charts (correlation, distributions, outcome balance)

• Clicks Train Models — all 4 algorithms train and evaluate

• Reviews comparison chart and metrics table

• Optionally overrides production model in Model Settings

• Patients now receive predictions from the selected model

9.3 Clinical Escalation Workflow

• Patient sends report to Admin (with optional message)

• Admin reviews report in Received Reports

• Admin forwards report to an assigned doctor with a note

• Doctor reviews in Forwarded Reports, adds clinical remark

• Patient sees remark in Notifications

• Patient can submit feedback with actual outcome → Admin retrains model

10. Installation & Deployment

10.1 Local Setup

- cd Intelligent_Diabetes_Risk_Predictor
- pip install -r requirements.txt
- python setup_data.py (downloads dataset)
- python train_initial.py (trains models, generates plots)
- python run.py (starts server at http://localhost:5000)

10.2 Demo Accounts

- Patient: username patient / password patient123 (no owner code)
- Doctor: username doctor / password doctor123 / owner code doctor2026
- Admin: username admin / password admin123 / owner code admin2026

10.3 Production Deployment (Render)

- Push project to GitHub.
- On Render: New + → Blueprint → select repository.
- Set OWNER_ADMIN_ACCESS_CODE and OWNER_DOCTOR_ACCESS_CODE environment variables.
- Optional: add DATABASE_URL for PostgreSQL persistence (Neon free tier).
- First build takes 5–8 minutes. App URL: https://intelligent-diabetes-risk-predictor.onrender.com
- Health check endpoint: /health
- Free tier sleeps after ~15 min idle; first load after sleep takes 30–60 seconds.

10.4 Docker

The Dockerfile uses python:3.12-slim, installs dependencies, copies app code with dataset and pre-trained models for offline-safe cold start. Gunicorn serves the app on port 10000 with 1 worker and 2 threads.

11. Testing

Unit tests are in the tests/ folder and run with: python -m pytest tests/ -v

A manual test plan covering all roles and features is in docs/TEST_PLAN.md.

Additional utility scripts exist for smoke checks, score verification, and form debugging.

12. Limitations & Future Enhancements

12.1 Current Limitations

- Trained on Pima Indians dataset — may not generalize to all populations.
- Only 8 features — not a comprehensive clinical assessment.

- Not a diagnostic tool — educational decision support only.
- Free hosting has cold-start delays and limited resources.
- No mobile app, SMS/email alerts, or EHR/hospital system integration.
- SQLite data on Render resets on redeploy without PostgreSQL.

12.2 Possible Future Improvements

- Larger and more diverse training datasets.
- SHAP/LIME for deeper model explainability.
- REST API for mobile clients.
- Multi-factor authentication for admin and doctor accounts.
- Email/SMS notifications for high-risk alerts.
- HL7/FHIR integration with hospital EHR systems.
- Real-time model monitoring and drift detection.
- Alembic database migrations for production schema management.

Disclaimer: This application provides an educational risk estimate based on statistical models trained on a public dataset. It does not replace professional medical advice, diagnosis, or treatment. Always consult qualified healthcare professionals.

Supervisor: Komal Khawar — komal.khawer@vu.edu.pk